

可逆的插件系统

Koishi 官方设计文档 · Shigma · 2023

可逆的插件系统

提示

本文将回答以下问题：

- 为什么我们需要可逆的插件系统？
- Cordis 是如何实现资源安全的？

Koishi 的一切都是从 Cordis 开始。但我想大部分 Koishi 的开发者都不知道 Cordis 是什么。如果让我来定义的话，Cordis 是一个**元框架 (Meta Framework)**，即一个用于构建框架的框架。

Cordis 的名字来源于拉丁语的心。我希望它能成为未来软件（至少是我开发的软件）的核心。

作为一个元框架，Cordis 并不耦合任何具体的领域或场景。它所提供的能力是大多数框架都不足为奇的——插件系统，但在这个系统背后却是大多数框架都没有达成的目标：可逆性。

背景介绍

引子：软件文明在退步吗？

我时常会觉得现代软件相比于曾经的软件存在着某种退步。在过去，我们用 C 这样的语言编写程序时，我知道 `open()` 会返回一个 `fd`，我知道 `malloc()` 会返回一个 `ptr`，我知道 `fork()` 会返回一个 `pid`。这些东西通常被称为**资源 (Resource)**。我也知道，为了编写可靠的程序，我应当在使用完这些资源后，调用对应的函数来回收它们。

而在如今，当我使用 Koa 时，我可以使用 `app.use()` 来注册一个中间件；当我使用 Vue 时，我可以使用 `app.component()` 来注册一个组件；当我使用 Node.js 时，我可以直接导入 `.node` 文件来加载使用 C++ 编写的模块。但很遗憾的是，Koa 不会告诉你如何取消这个中间件，Vue 不会告诉你如何卸载这个组件，Node.js 甚至会永久占用这个 `.node` 文件。

你当然可以说这是软件发展的结果：底层 API 被妥善地封装了，开发者不再需要关心这些细节。但被封装后的资源仍然是资源，它们仍然有着被回收的需求。或许对于每一个具体的场景，我们都可以找到一个解决方案，或者给出我们不需要回收资源的理由，但面对一个复杂的、未知的应用，如果你想要回收资源而它又没有提供相应的 API，最好的办法就只有重启了。

事实上，封装也根本不是导致这种现象的原因。面对不当使用指针引发的内存安全问题，无论是 C++ 的智能指针、Java 的垃圾回收机制，还是 Rust 的所有权系统，都提供了对指针的封装。这些封装不仅不会导致内存泄露，反而通过提高易用性减少了开发者的心智负担。

有了正面的例子，我们就可以知道，这种退步实际上只是特定领域中呈现出的趋势（在 JavaScript 和 Python 这种高级语言中尤为明显）。或许是人们认为重启过于方便了，因此一些框架的开发者们

已经完全不考虑回收资源的需求了。但现代软件就如同摩天大楼，一旦某一层缺失了支撑，在其上的一切都会变得摇摇欲坠。好在我们或许有办法改善这一切。

可逆性的优势

可逆性 (Disposability)，即回收资源的能力，可以为软件带来以下好处：

可组合性 (Composibility)。很多软件很喜欢用「模块化」「插件」这样的词，这显然是来自现实世界的概念。然而现实中的模块也应当是可拆卸的，现实中的插件也应当是可以拔出的。不可逆的软件即便进行了模块化，也只会随着时间推移而变得更加臃肿。此外，可逆性可以让我们更好地理解模块之间的依赖关系，从而更好地促成解耦。这一点我们稍后会进一步讨论。

可靠性 (Reliability)。当软件的规模增加时，可逆性可以确保软件所使用的内存和其他资源都在可控范围内（内存安全其实是资源安全的一种特殊情况）。同时，由于可逆性也意味着可追踪性，即便某个模块出现了资源泄露，我们也可以快速定位错误的来源。

可访问性 (Availability)。一个拥有众多功能的软件，如果没有提供可逆的 API，那替换任何一个组件都意味着整个重启。重启期间，那些本可以不受影响的服务也被迫下线。但如果其中的每个组件都是可逆的，我们就可以在保证其他功能持续运行的情况下替换掉任何一个组件，甚至可以滚动更新整个程序自身。实现了可逆性后，软件将显著降低由于故障和更新带来的额外开销。

可逆的 Koishi

相比上面这些可能有些晦涩的概念，以 Koishi 作为更具体的例子或许更有说服力。

可逆的 Koishi 是指，对于任何一个 Koishi 实例，任意进行加载和卸载插件操作后，最终行为仅与最终启用的插件相关；与中间是否重复加载过插件、插件之间的加载或卸载顺序都无关。你也可以简单理解为「路径无关」。这里的相关和无关具体包括：

- 任意次加载并卸载一个插件后，内存占用不会增加。
- 任意次加载并卸载一个插件后，不会残留对其他插件的影响。
- 如果插件之间有依赖关系，依赖的插件会自动在被依赖的插件之后加载，并自动在被依赖的插件之前卸载，即确保插件的生命周期由依赖关系而非加载顺序决定。

实现了可逆性的 Koishi 项目将获得以下优点：

- **热重载**：由于插件的副作用会在卸载时回收，Koishi 的所有插件都将可以在运行时加载、卸载和重载。这显著降低了用户的开发和更新成本，并大幅提高了 Koishi 应用的 SLA。
- **异步加载**：由于插件的加载顺序由依赖关系决定，因此插件的代码可以被异步地加载，而不需要担心加载顺序对可用性的影响。这将显著提高 Koishi 的启动速度。
- **可追踪**：由 Koishi 插件注册的指令和中间件、监听的事件、提供的本地化、扩展的页面、抛出的错误都可以被明确地追踪来源。这有利于在大型项目中快速定位问题。

如今，Koishi 已经有超过 3000 个插件，其中的依赖错综复杂。而即使是在这个规模下，Koishi 仍然能够妥善处理所有插件的加载、卸载和更新。这一切都得益于 Cordis 的可逆性。

实现原理

说了这么多好处，可逆性真的可以实现吗？答案是肯定的。在这一节中，我们将会从数学的角度来探讨可逆性的实现原理。你会发现，任何语言都可以实现自己的 Cordis。

可逆的副作用

函数式编程中有着纯函数的概念——给定相同的输入总是给出相同的输出。然而，现实中的程序往往要与各种各样的副作用打交道。对于这种情况，我们可以对函数进行“纯化”——将它的副作用转化为参数和返回值的一部分即可。考虑下面的函数：

$$f_{\text{impure}} : X \rightarrow Y$$

假设它含有副作用，我们把所有可能的副作用用类型 $\mathcal{C}$ 封装起来，则该函数可以被转化为：

$$f : \mathcal{C} \times X \rightarrow \mathcal{C} \times Y$$

此时我们得到的就一个纯函数，它接受 $\mathcal{C}$ 和参数，返回修改过的 $\mathcal{C}$ 和返回值。

如果忽略 f 本身的入参和出参，只考虑副作用，那么可以定义函数空间 $\mathfrak{F} = \mathcal{C} \rightarrow \mathcal{C}$ 。其中的任何一个函数 $f : \mathfrak{F}$ 都是 $\mathcal{C}$ 到自身的变换，不难看出它们在函数结合 $\circ$ 下构成么半群：

1. 封闭性： $f \circ g$ 也是 $\mathcal{C}$ 到自身的变换。
2. 结合律： $(f \circ g) \circ h = f \circ (g \circ h)$ 。
3. 单位元： 存在 id ，使得 $f \circ \text{id} = \text{id} \circ f = f$ 。

进一步，我们还希望 f 的副作用是可以回收的。换言之，我们额外要求 f 存在逆元 f^{-1} ，此时 $\mathfrak{F}$ 就构成一个群。但仅仅知道函数可逆并不能帮助我们找到它的逆，我们需要在书写这个函数时一并写出它的回收方法。因此我们引入 `effect` 函子，使这个函数返回一个新的函数，这个函数可用于回收此次调用的副作用：

$$\begin{aligned} \text{effect} & : \mathfrak{F} \rightarrow \mathcal{C} \times \mathfrak{F} \rightarrow \mathcal{C} \times \mathfrak{F} \\ \text{effect} & = f \mapsto (c, h) \mapsto (f(c), h \circ f^{-1}) \end{aligned}$$

可以证明 `effect` 是一个 $\mathfrak{F}$ 到 $\mathcal{C} \times \mathfrak{F}$ 的同态：

$$\begin{aligned} \text{effect } (f \circ g) (c, h) &= ((f \circ g)(c), h \circ (f \circ g)^{-1}) \\ &= (f(g(c)), h \circ g^{-1} \circ f^{-1}) \\ &= \text{effect } f (g(c), h \circ g^{-1}) \\ &= (\text{effect } f \circ \text{effect } g) (c, h) \end{aligned}$$

下面是一个例子：

```
function serve(port: number) {  
  const server = createServer().listen(port)  
  return () => server.close()  
}  
  
const dispose = serve(80)    // 监听端口 80  
dispose()                    // 回收副作用
```

在这个例子中，`serve()` 函数将会创建一个服务器并且监听 `port` 端口。同时，调用该函数也会返回一个新的函数，用于取消该端口的监听。

提示

你可能很难将这段 TypeScript 代码与上面的数学定义对应起来，这是因为 TypeScript 并不是一个纯函数式语言。具体而言，这段代码以如下的方式建立对应关系：

- $\mathcal{C} \times \mathfrak{F}$ 对应着全局环境（我们稍后会提到全局环境的坏处，但不影响这里的理解）
- `port` 对应于上面的 X ，由于我们可以使用柯里化，所以在数学模型中并不需要考虑它

为什么需要引入这个 `effect` 和 $\mathcal{C} \times \mathfrak{F}$ 呢？它的作用是将副作用从函数的返回值中分离出来，从而实现副作用的回收。只需定义 `restore` 变换（不难发现它确实是 `effect` 的逆操作）：

$$\begin{aligned}\text{restore} &: \mathcal{C} \times \mathfrak{F} \rightarrow \mathcal{C} \times \mathfrak{F} \\ \text{restore} &= (c, h) \mapsto (h(c), \text{id})\end{aligned}$$

现在你就可以使用 `restore()` 来回收副作用了：

```
function serve(port: number) {
  const server = createServer().listen(port)
  collectEffect(() => server.close())
}

serve(80)           // 监听端口 80 并记录副作用
serve(443)          // 监听端口 443 并记录副作用
restore()           // 回收所有副作用
```

当副作用被记录到全局环境时， $\mathcal{C} \times \mathfrak{F}$ 也就变成了一个更大的 $\mathcal{C}$ 。我们便可以这样定义：

$$\mathcal{C}_1 = \mathcal{C} \times \mathfrak{F} = \mathcal{C} \times (\mathcal{C} \rightarrow \mathcal{C})$$

下文中我们将直接使用 $\mathcal{C}$ 来表示 $\mathcal{C}_1$ 。

上下文与插件

在上面的示例中，我们并没有显式地写出 $\mathcal{C}$ 参数和返回值。可以认为对 $\mathcal{C}$ 的变换存在于所有全局函数的闭包中。这种设计广泛存在于各种组合式框架（尤其是像 React 这样的前端框架），但一些缺陷使其并不适合插件化和规模化的场景。

首先，所有插件都使用相同的全局函数，意味着不同插件的副作用完全无法区分，因此只能重启整个应用而无法细粒度地控制具体的插件；其次，这种设计意味着全局函数并不纯，因此一旦项目中出现了多例的依赖，整套系统的可靠性就会完全失效！

引入显式 $\mathcal{C}$ 变换会降低应用的可读性，忽略显式 $\mathcal{C}$ 变换又存在上述缺陷。那么有没有办法在不增加心智负担的同时编写可靠的插件呢？Cordis 通过上下文对象给出了完美的解决方案。

上下文对象是一个插件中唯一的可变部分，它同时担任了 $\mathcal{C}$ 参数和返回值的角色。在上面的示例中引入上下文对象，就得到了熟悉的 Koishi 插件：

```
function serve(ctx: Context, config: Config) {
  const server = createServer().listen(config.port)
  ctx.on('restore', () => server.close())
}
```

相应地，我们使用 `ctx.plugin()` 来加载插件：

```
ctx.plugin(serve, { port: 80 })
```

这看起来只是把函数和参数调换了个位置，但实际上外侧的 `ctx` 跟插件内部拿到的 `ctx` 并不是同一个值。当一个插件被加载时，将会从当前上下文对象上派生出一个新的上下文实例。子级上下文将管理插件内的全部副作用，而插件整体将作为一个副作用被父级上下文收集。可以将上下文比作一个副作用的插座，而副作用就是上面的插头。当上下文被卸载时，它将会将所有的副作用一一回收。而插件就是连接到另一个插座的插头，管理着子级上下文的全部副作用。

除了 `ctx.plugin()` 外，上下文对象上还有许多 API，它们几乎都是某个函数的可逆化版本。例如 `ctx.on()` 是添加监听器的可逆化，`ctx.command()` 是注册指令的可逆化。这样一来，开发者只需要调用 `ctx` 上的方法，就可以确保插件的作用是可逆的。

这种设计同时解决了上述两个缺陷，并且完全不会带来额外的心智负担。在大多数的插件场景下，开发者甚至完全不需要手动监听 `restore` 事件，就能编写出可逆的插件。换句话说，只要框架的能力够强，将某一场景的所有 API 都通过可逆的方式提供，插件开发者就可以在完全不理解这套理论的情况下自然地编写出可逆的插件。

高阶的资源

从上面的视角下，我们或许能对资源有一个更深刻的认识。任何一个函数，它要么是纯函数，要么存在副作用，而这个副作用本身就是函数对外占用的资源。这些资源可以是底层的内存、文件、进程，也可以是上层的各种封装。提供了完整回收副作用的能力，就可以称为是「资源安全」的。

那如果一个插件提供了 API 给别的插件使用，这个插件占用资源了吗？是的。因为要想让别的插件使用，别的插件就必然需要访问你提供的 API (而不是别人提供的)。无论这种访问逻辑是通过什么实现的，提供 API 的插件都需要占用该访问资源。

进一步，如果这个 API 本身还存在副作用，那提供此 API 的插件其实占用的是一种能占用资源的资源，一种高阶资源。就如同高阶函数一样，我们的 $\mathcal{C}$ 也可以是高阶的：

$$\begin{aligned}\mathcal{C}_1 &= \mathcal{C}_0 \times (\mathcal{C}_0 \rightarrow \mathcal{C}_0) \\ \mathcal{C}_2 &= \mathcal{C}_1 \times (\mathcal{C}_1 \rightarrow \mathcal{C}_1) \\ &\dots \\ \mathcal{C}_{n+1} &= \mathcal{C}_n \times (\mathcal{C}_n \rightarrow \mathcal{C}_n)\end{aligned}$$

在 Cordis 中，插件之间默认情况下不存在先后关系。换句话说，默认任何两个插件的执行顺序都是可以交换的。如果你想要表达插件之间的依赖关系，则需要通过 **服务 (Service)** 来实现。服务用一个字符串表示，可以被插件提供 (provide) 或注入 (inject)。

Cordis 通过其自身的机制确保提供任何一对提供 / 注入同名服务的插件的生命周期都是包含关系。此外，Cordis 还提供了服务隔离的概念，开发者可以为任何一个服务名称创建隔离上下文，使其内部和外部的插件对于该服务名称无法相互感知和访问。

畅想：可组合性的本质

很多人谈论可组合性，主要说的是解耦，也就是将代码拆解到不同函数、不同模块的能力。但其实我们编写的代码并不是静态的，可组合性可以在更多的维度上定义：

- 逻辑可组合性：代码自身的解耦能力 (常见的理解方式)。
- 时间可组合性：代码可以被同时加载、可以被回收副作用的能力 (本文主要介绍的部分)。

- 空间可组合性：代码之间能够有效声明和隔离依赖关系的能力。

我希望借助 Cordis 这个框架，勾勒出软件文明的一个未来。在这个未来，人们可以按照需求背后的本质逻辑，组合出高效、可靠、易于开发和维护的软件。

畅想：在语言层面确保资源安全

Rust 声称自己在语言层面确保了内存安全 (具体是不是这里不做讨论)，那么 Cordis 能否在确保资源安全呢？很遗憾，目前并不能。开发者只需设置几个全局变量、或者调用一些未被封装过的 API，就可以绕过 Cordis 的保护机制。但这并不意味着 Cordis 是无用的。如果未来我们将所有的底层 API 都封装起来，并确保用户只能通过上下文调用，那么 Cordis 就可以确保资源安全了。

一种更好的思路是直接从语言层面加以设计。例如可以将全局变量的访问和一些底层 API 视为“不安全”的，那么一个不含 `unsafe` 关键字的代码片段就可以被证明资源安全的了。我们还可以在编译期间检查出所有的资源安全问题，而不需要等到运行时才发现。少数函数式编程语言实现了 Algebraic Effects，可以实现类似资源安全的概念。不过受限于函数式语言本身的特性，要让主流的软件开发者接受这种编程范式还需要很长的时间。

相比较而言，Cordis 在设计上能够与主流的 OOP 语言完美结合，并且不需要重构整套系统。任何特定领域的框架都可以通过 Cordis 来实现可逆性，而对应领域的插件开发者也可以在不了解任何数学知识的情况下编写可逆的插件。这种渐进性是 Cordis 的一大优势。